

Design-Space Exploration For Compute-Near-Memory Systems

Clément Fournier, João P. C. de Lima, Asif Ali Khan, Hamid Farzaneh,
Jerónimo Castrillón
TU Dresden

Cinnamon for Design-Space Exploration

Today

The space of possible programs is too large
to enumerate.

How do we find a good one?

Today

The space of possible programs is too large to enumerate.

How do we find a good one?

- | | |
|-------------------------------|------------|
| 1. The design space | notebook 1 |
| 2. Evaluating a configuration | notebook 2 |
| 3. Searching the space | notebook 3 |

Today

The space of possible programs is too large to enumerate.

How do we find a good one?

- | | |
|-------------------------------|------------|
| 1. The design space | notebook 1 |
| 2. Evaluating a configuration | notebook 2 |
| 3. Searching the space | notebook 3 |

Open the notebooks now



<https://tinyurl.com/tuto-dse>

Binder takes a minute or two to start.

The hardware

Compute near memory

UPMEM DPUs in DRAM ranks

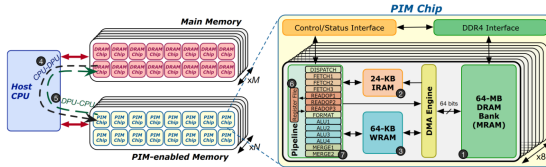


Fig. 8. An overview of the UPMEM architecture [43].

HBM-PIM SIMD units per bank

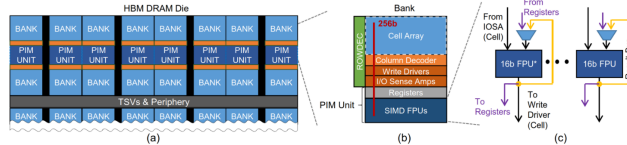
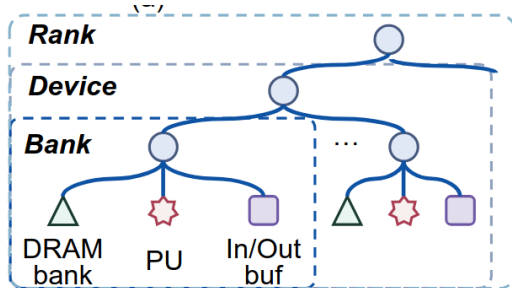


Fig. 11. Samsung's PIM-HBM (a) HBM die organization (b) Bank coupled with a PIM unit (c) PIM unit data path [7].

Figures: Khan et al., 2024.

What they have in common

- Many small compute nodes
- Each owns a slice of memory
- A hierarchy of levels
- A narrow interface to other memory spaces
- Programmable



Running a kernel on such a system means deciding how to distribute it over the nodes and memories.

Figure: Xie et al., HPCA 2025.

The problem

Running example: $y = Ax$

$$y_m = \sum_k A_{m,k} x_k$$

for $m \in M$, $k \in K$

The two loop dimensions behave differently:

- M is **parallel**: rows are independent
- K is a **reduction**: splitting it produces partial sums that must be added up

Running example: $y = Ax$

$$y_m = \sum_k A_{m,k} x_k$$

for $m \in M$, $k \in K$

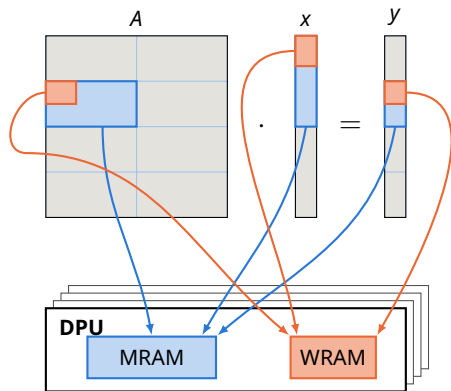
The two loop dimensions behave differently:

- M is **parallel**: rows are independent
- K is a **reduction**: splitting it produces partial sums that must be added up

In Cinnamon's MLIR input:

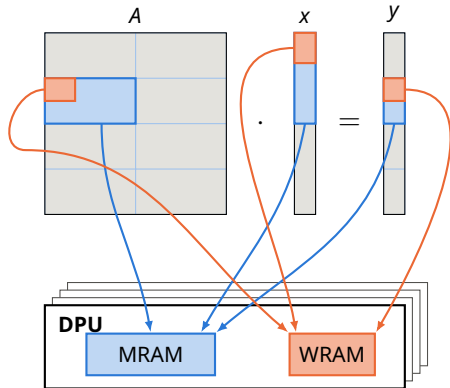
```
#upmem = #upmem.platform<type = v1A,  
                                dpus = 64, tasklets = 24>  
  
func.func @gemv(%A: tensor<1024x1024xi32>,  
                %x: tensor<1024xi32>) {  
    attributes {cinm.available_platforms = [#upmem]} {  
        cinm.compute {  
            %y = cinm.op.gemv %A, %x  
        }  
    }  
}
```

Tiling to the memory levels



MRAM tile: $M.\text{mram} \times K.\text{mram}$, one per (DPU, tasklet)
WRAM tile: $M.\text{wram} \times K.\text{wram}$, copied in one at a time
order: which dimension is spread across DPUs first

Tiling to the memory levels



MRAM tile: $M.mram \times K.mram$, one per (DPU, tasklet)

WRAM tile: $M.wram \times K.wram$, copied in one at a time
order: which dimension is spread across DPUs first

Each choice must satisfy

- tiles divide the extents: $M.mram \mid 1024, \dots$
- WRAM tiles divide MRAM tiles: $M.wram \mid M.mram, \dots$
- one tile per worker: $\frac{1024}{M.mram} \cdot \frac{1024}{K.mram} = dpus \cdot tasklets$
- what the tasklets hold fits in MRAM and WRAM
- tiles are multiples of the 8-byte DMA granule

The hardware knobs are not separate

How many DPUs? How many tasklets?

It is tempting to fix these first and tile afterwards, but:

- the tile count must equal the worker count:
$$\prod(\text{extent}/\text{tile}) = \text{dpus} \cdot \text{tasklets}$$
- more tasklets $\Rightarrow$ smaller tiles $\Rightarrow$ more launch overhead per byte
- more DPUs $\Rightarrow$ more transfer parallelism, more partial results to gather

Measured on the small space

best latency with `tasklets` =

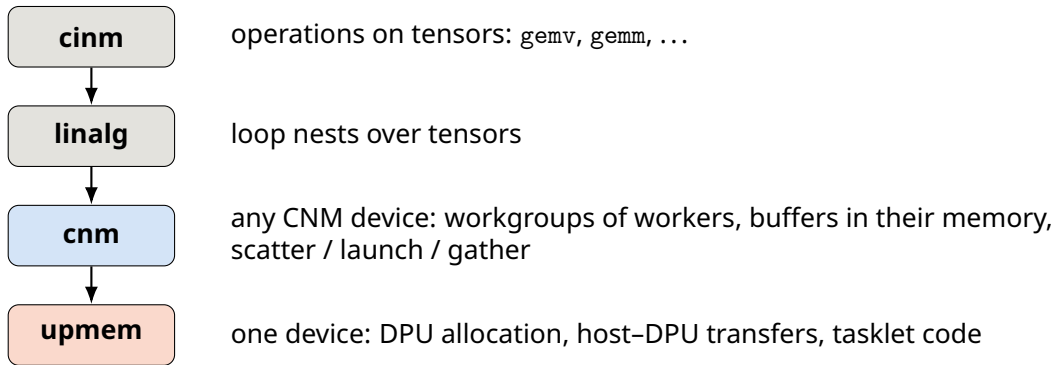
1	2	4	8	16
8.0	4.2	2.4	1.6	7.8

(ms). Using the most tasklets is $5\times$ slower than the best.

For 4096² on 32 ranks, the best configuration uses **1 tasklet** on all 2048 DPUs.

So hardware and tiling parameters have to be chosen together.

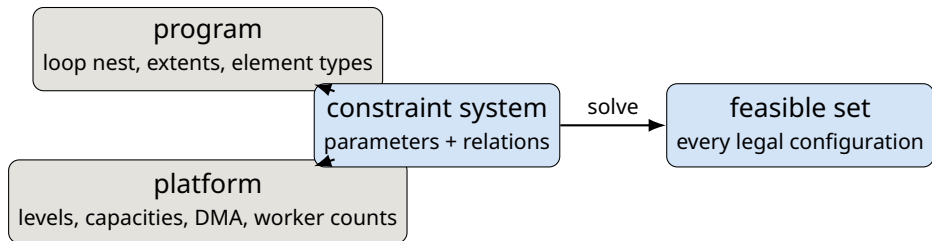
Cinnamon



other targets lower from cnm the same way

Each step down needs decisions: how many workers, how to tile, how to distribute. These are the parameters of the design space.

From the program to a space



Cinnamon generates the parameters and the constraints between them from the program and the platform description, and enumerates the feasible configurations with a constraint solver.

Notebook 1

The design space

The program, its
parameters and the
constraints between them.

🕒 7 min

Evaluating a configuration

Lowering one configuration

Given a configuration, Cinnamon compiles the program:

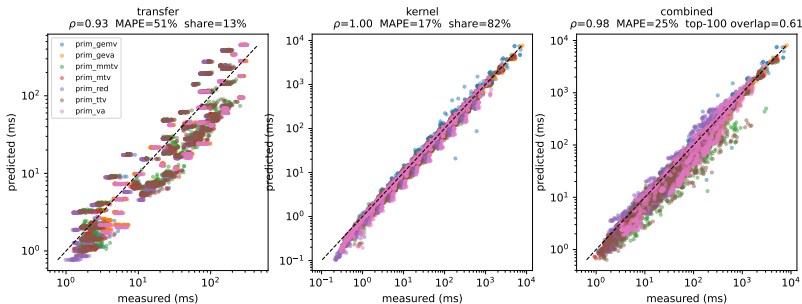
- **cinm** → **linalg**: using builtin operators
- **linalg** → **cnm**: the loop nest is split over a workgroup of $\text{dpus} \times \text{tasklets}$ workers. Inputs are scattered into per-worker MRAM buffers, the result is gathered back.
- **On-device tiling**: each worker's kernel loops over its MRAM tile, copying WRAM-sized pieces into the scratchpad
- **cnm** → **upmem**: transfers between host and DPUs, DPU allocation, the tasklet code

parameter	used at
dpus, tasklets	workgroup shape
M.mram, K.mram	cnm buffer shapes
order	scatter / gather maps
M.wram, K.wram	kernel loops

Estimating the latency

The estimate for one configuration is the sum of

- the **DPU kernel**, from a cycle-accurate DPU simulator (or a faster approximation of it)
- the **launch overhead**, calibrated on hardware
- **host work**: rearranging data before transfers, reductions on the host
- **transfers** to and from the DPUs



Notebook 2

Evaluating a configuration

Pick a configuration, follow its lowering, simulate it, and see where it ranks.

🕒 8 min

Searching the space

Enumerating is not an option

	feasible	one core, fast simulator	
1024 ² gemv, 1 rank	3 234	3 min	oracle in notebook 2
4096 ² gemv, 32 ranks	26 238	1.5 h	enumerated on 14 cores
2048 ² gemm, 32 ranks	2 821 332	47 h	not enumerated

Every configuration a search visits costs one simulation, so a search is judged by how good a configuration it finds for a given number of simulations.

Search strategies

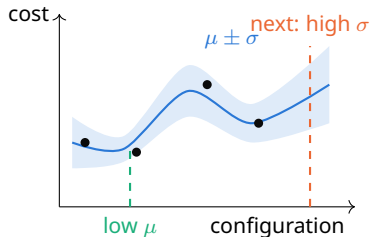
strategy	how the next configuration is chosen	uses past results
exhaustive	every feasible one, in order	no
random	uniformly at random	no
local search, simulated annealing	a neighbour of the current one	the current one
genetic algorithm	recombining good ones	a population
Bayesian optimisation	where a model of the cost looks promising	all of them

The more a strategy learns from past results, the more it can save on simulations, and the more it costs to decide what to simulate next.

Bayesian optimisation in one slide

1. **Initial sample:** simulate a few random configurations
2. **Surrogate:** fit a model to the results. For any configuration it predicts a cost μ and an uncertainty σ usually a Gaussian process; in Cinnamon an ensemble of small neural networks
3. **Acquisition:** score candidates by $\mu - \kappa\sigma$, so both a low predicted cost and a high uncertainty make a candidate attractive
4. **Simulate** the best-scoring candidate, add it to the results, go to 2

The search only proposes feasible configurations, so no simulation is spent on a configuration that cannot be compiled.



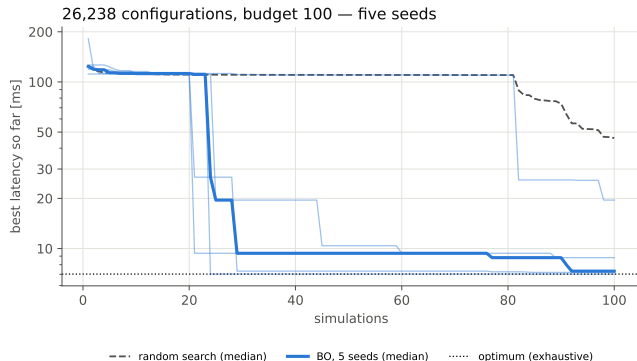
Notebook 3

Searching the space

Bayesian optimisation with
40 and 100 simulations,
several seeds, and a larger
space. 🕒 8 min

Wrap-up

Where enumeration was never an option



26 238 configurations, budget 100

	ms	rank
optimum (oracle)	7.0	1
Bayesian, best of 5 seeds	7.0	1
Bayesian, median seed	7.3	8
Bayesian, worst seed	19.5	82
random search, median	46	
median configuration	125	

Three of five seeds end within 10 % of the optimum, after simulating 0.4 % of the space. Only 10 of the 26 238 configurations are that close.

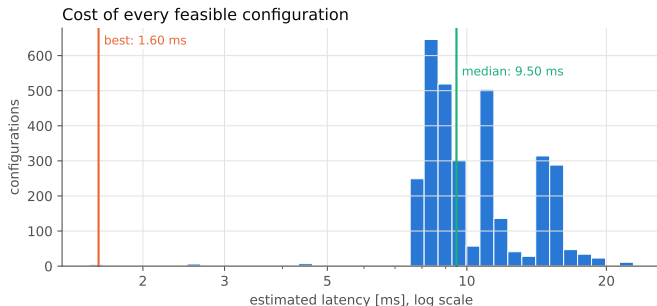
Take-aways

1. The design space follows from the program and the platform: the parameters and the constraints between them can be generated rather than written by hand.
2. Hardware parameters (DPUs, tasklets) and tiling parameters have to be chosen together. The best number of tasklets changes with the problem size.
3. On a small instance an exhaustive run shows what the landscape looks like, and gives the optimum to compare searches against.
4. Search results depend on the random seed: compare several runs, not one.

Cinnamon: github.com/tud-ccc/Cinnamon this tutorial: [repository]

Questions?

We simulated all 3 234

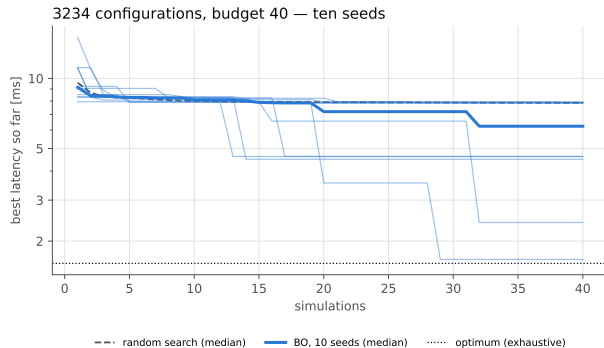


best	1.60 ms
1st percentile	6.25 ms
median	9.50 ms
worst	21.8 ms

within 10 % of best	6
within 1.5×	11
within 2×	19

The best 0.3 % of configurations are 5× faster than the median.

What a budget buys



Budget 40, ten seeds

seed	best [ms]	rank
166	1.60	1
197	1.61	2
104	1.64	4
290	1.67	6
135	2.41	11
73	7.85	45
259	7.87	66
228	7.89	80
42	7.90	103
321	7.91	107

4 of 10 seeds find one of the best configurations (random search: 14 %).
With 100 simulations, 4 of 5 find the optimum.